

Design and Implementation of an Optimal Binary Search Tree Using Dynamic Programming

1. Introduction

This project implements an Optimal Binary Search Tree (OBST) using Dynamic Programming for a university Design and Analysis of Algorithms assignment. The goal is not only to produce a working program, but also to demonstrate the algorithmic reasoning behind the solution: state definition, recurrence, bottom-up table construction, reconstruction of the tree, verification of the expected cost, complexity analysis, and experimental measurement.

A Binary Search Tree can store the same sorted set of keys in many different shapes. Since a successful search follows a path from the root to the target key, keys closer to the root require fewer comparisons. If all keys are equally likely, a balanced tree is usually a reasonable goal. However, when some keys are searched more frequently than others, the most useful tree is the one that minimizes probability-weighted search cost. This is the motivation for the Optimal BST problem.

2. Problem Definition

The input is a sorted set of unique keys:

$$K = \{k_1, k_2, \dots, k_n\}$$

and successful-search probabilities:

$$p = \{p_1, p_2, \dots, p_n\}$$

Each p_i is non-negative, and all probabilities sum to approximately 1. Only successful searches are considered in this project.

The output is a Binary Search Tree that minimizes:

$$\text{sum}(p_i * \text{comparisons}_i)$$

The root depth is defined as 0. Therefore, the number of comparisons for a successful search is:

$$\text{comparisons}_i = \text{depth}_i + 1$$

3. Dynamic Programming Formulation

Let $C[i,j]$ represent the minimum expected successful-search cost for keys k_i through k_j in mathematical 1-based notation.

Base cases:

$$\begin{aligned} C[i,i] &= p_i \\ C[i,i-1] &= 0 \end{aligned}$$

The first base case means that a subtree containing one key has one successful comparison weighted by that key's probability. The second base case represents an empty subtree, which contributes no search cost.

For an interval $[i,j]$, every key r where $i \leq r \leq j$ is tried as the root. If r is chosen, then keys $i..r-1$ form the left subtree and keys $r+1..j$ form the right subtree. Both subtrees become one level deeper under the selected root, so the total probability weight of the interval must be added.

The recurrence is:

$$C[i,j] =$$

$$\text{sum}(p_s \text{ for } s = i..j) \\ + \min \text{ over } r = i..j (C[i,r-1] + C[r+1,j])$$

The implementation stores the same state with 0-based Python lists. Therefore, mathematical $C[i,j]$ corresponds to Python `cost[i-1][j-1]`.

The project also stores a `root[i][j]` table. The cost table gives the optimal value, but the root table records which root produced that value. Without the root table, the program could report the minimum cost but could not efficiently reconstruct the final tree.

4. Algorithm / Pseudocode

```
OBST(keys, p):
    n = number of keys

    create cost[n][n]
    create root[n][n]
    build prefix sums

    for i = 0 to n-1:
        cost[i][i] = p[i]
        root[i][i] = i

    for length = 2 to n:
        for i = 0 to n-length:
            j = i + length - 1
            cost[i][j] = infinity
            total_weight = weight(i,j)

            for r = i to j:
                left_cost = cost[i][r-1] if r > i else 0
                right_cost = cost[r+1][j] if r < j else 0
                candidate = left_cost + right_cost + total_weight

                if candidate < cost[i][j]:
                    cost[i][j] = candidate
                    root[i][j] = r

    tree = reconstruct(root, 0, n-1)
    return cost[0][n-1], cost, root, tree
```

The table is filled by increasing interval length. This guarantees that every subproblem needed for a larger interval has already been computed.

5. Implementation

The implementation is divided into small modules:

File	Purpose
src/obst.py	DP algorithm, validation, reconstruction, tree metrics
src/models.py	TreeNode and OBSTResult dataclasses
src/conventional_bst.py	Standard insertion-order BST baseline
src/input_parser.py	Text and CSV input parsers
src/display.py	Tables and ASCII tree formatting
src/experiments.py	Deterministic timing experiments
src/main.py	Command-line interface

Input validation rejects empty input, duplicate keys, unsorted keys, negative probabilities, malformed numeric values, and probability sums that are not approximately 1. The selected tolerance is 1e-6, which is sufficient for ordinary floating-point rounding differences in decimal inputs.

Prefix sums are used with:

```
prefix[0] = 0
prefix[i+1] = prefix[i] + p[i]
weight(i,j) = prefix[j+1] - prefix[i]
```

This makes the probability sum for any interval available in O(1) time.

Without prefix sums, repeatedly summing inside the nested DP loops would add unnecessary work.

Sample Case

The required sample input is:

```
5
10 0.10
20 0.20
30 0.40
40 0.20
50 0.10
```

The computed DP cost table is:

	10	20	30	40	50
10	0.1000	0.4000	1.1000	1.5000	1.8000
20	-	0.2000	0.8000	1.2000	1.5000
30	-	-	0.4000	0.8000	1.1000
40	-	-	-	0.2000	0.4000
50	-	-	-	-	0.1000

The computed root table displays actual key values:

	10	20	30	40	50
10	10	20	30	30	30
20	-	20	30	30	30
30	-	-	30	30	30
40	-	-	-	40	40
50	-	-	-	-	50

The optimal root for the full interval is 30, so the reconstructed tree is:

```

30
|-- L: 20
|  |-- L: 10
|  |-- R: 40
|  |-- R: 50

```

Depth and comparison table:

Key	Probability	Depth	Comparisons
10	0.10	2	3
20	0.20	1	2
30	0.40	0	1
40	0.20	1	2
50	0.10	2	3

Expected-cost verification:

$$\begin{aligned}
 &0.40 * 1 \\
 &+ 0.20 * 2 \\
 &+ 0.20 * 2 \\
 &+ 0.10 * 3 \\
 &+ 0.10 * 3 \\
 &= 1.8
 \end{aligned}$$

The program independently recomputes this value from the reconstructed tree and checks that it matches the DP result:

Minimum expected search cost (DP)	1.800000
Expected cost from reconstructed tree	1.800000
Verification result	success

6. Experimental Results

Experiments are generated and timed by the program. The final values in this section match experiments/results.csv.

n	Mean execution seconds	Minimum expected cost	Seed	Repetitions
5	0.00002722	1.79467990	2031	5
10	0.00005962	2.56711511	2036	5
20	0.00028114	3.03411892	2046	5
50	0.00266794	4.38649218	2076	5
100	0.01913680	5.31942274	2126	5

The experiment generator uses deterministic seeds. For each n, it creates sorted unique keys, generates positive random weights, and normalizes those weights so probabilities sum to 1. Input generation and terminal printing are not included in the measured algorithm timing.

The measured running time increases as n grows, which is consistent with the expected cubic behavior. In particular, increasing from n = 50 to n = 100 doubles the input size. The theoretical cubic model predicts approximately:

$$2^3 = 8 \text{ times more work}$$

The measured time changes from 0.00266794 seconds to 0.01913680 seconds:

0.01913680 / 0.00266794 approximately 7.17

This is close to the theoretical factor of 8. Exact ratios are not expected because Python interpreter overhead, operating system scheduling, CPU caching, timer noise, and small sample sizes can affect measured timings.

7. Comparison with a Conventional BST

The conventional BST baseline inserts the same keys in the supplied input order. This is important because a conventional BST is insertion-order dependent. Since the assignment input is sorted, direct insertion intentionally creates a highly unbalanced tree:

```
10
  -- R: 20
    -- R: 30
      -- R: 40
        -- R: 50
```

The comparison uses the same probabilities as the OBST. Therefore, the difference in expected cost comes from tree shape rather than from changing the input data.

For the sample input:

Tree	Expected successful-search cost	Unweighted average depth
Optimal BST	1.800000	1.200000
Conventional sorted-insertion BST	3.000000	2.000000

This shows that the OBST places high-probability keys closer to the root and achieves a lower expected number of comparisons.

8. Complexity Analysis

Time complexity is $O(n^3)$.

There are $O(n^2)$ intervals $[i,j]$ in the DP table. For each interval, the standard algorithm tries each possible root inside the interval, which is up to $O(n)$ choices. Therefore:

$$O(n^2) * O(n) = O(n^3)$$

Space complexity is $O(n^2)$.

The algorithm stores two $n \times n$ tables:

```
cost[n][n]
root[n][n]
```

The prefix sum array uses $O(n)$ space, and the reconstructed tree uses $O(n)$ node space. These do not change the total asymptotic space complexity.

9. Discussion

An OBST is not necessarily the same as a balanced BST. A balanced BST focuses on height, while an OBST focuses on probability-weighted search cost. If a key has a high search probability, the optimal solution may move it closer to the root even if the final tree is not perfectly balanced.

The conventional BST comparison is useful because it shows the weakness of simple insertion-order construction. With sorted input, the conventional BST can become a chain, giving poor search depth for many keys. The OBST avoids this by evaluating all possible roots for each interval.

The implementation follows the required standard $O(n^3)$ DP method. It does not use Knuth optimization or another advanced optimization, because the assignment focuses on the basic Dynamic Programming formulation.

One limitation is that this project handles only successful-search probabilities. Some textbook versions of OBST also include unsuccessful-search probabilities for gaps between keys. That extended model is outside the scope of this assignment.

10. Conclusion

This project demonstrates a complete Dynamic Programming solution to the Optimal Binary Search Tree problem. The cost table computes the minimum expected search cost, the root table records the choices needed for reconstruction, and the final tree is verified by recalculating expected cost from actual node depths.

The sample case produces the expected minimum cost of 1.800000. The conventional BST comparison produces a higher expected cost of 3.000000, showing why probability-aware tree construction matters.

11. References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, Dynamic Programming chapter.
- Donald E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching.